OpenHands

Anti-Slop

High Quality Agentic Coding

Windows

An error has occurred. To continue:

Press Enter to return to Windows, or

Press CTRL+ALT+DEL to restart your computer. If you do this,
you will lose any unsaved information in all open applications.

Error: 0E : 016F : BFF9B3D4

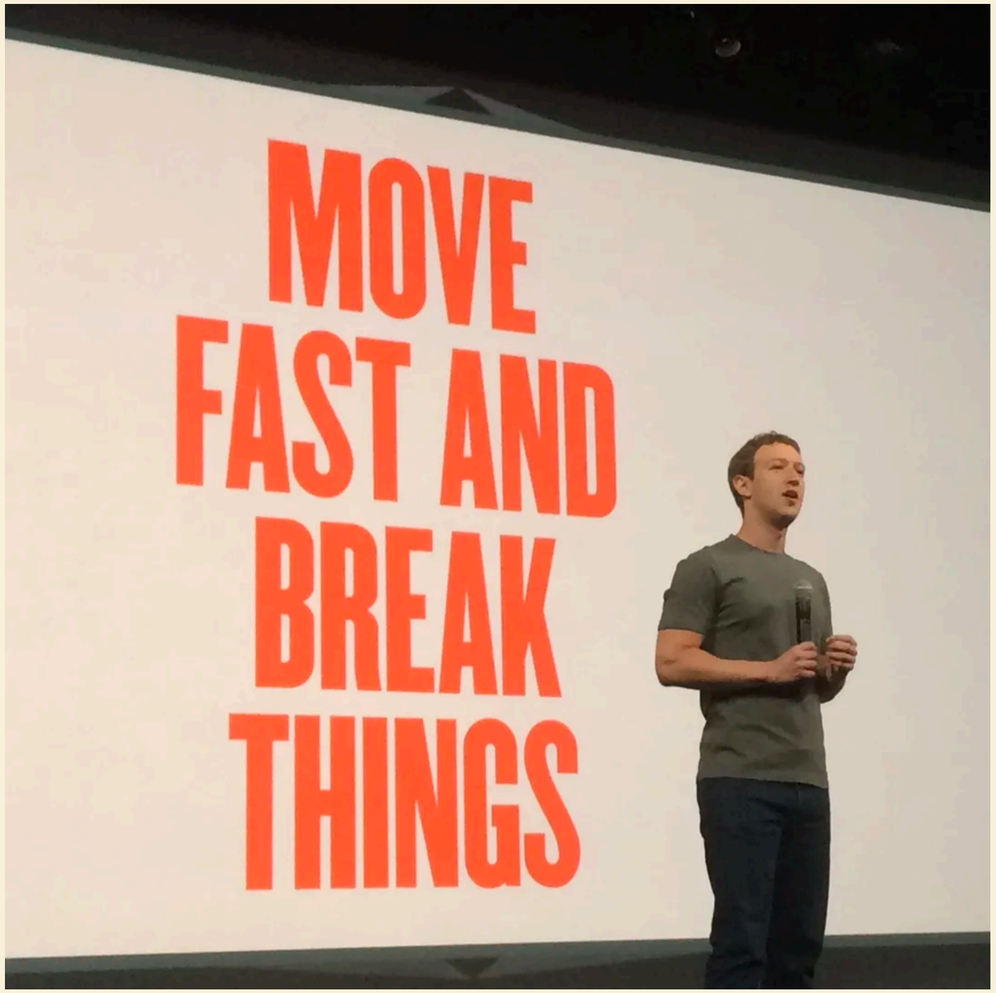
Press any key to continue _

Blue Screen of ??

Death

OpenHands

Software Should Work



Worst slogan ever: 2004-2014

Worst slogan ever: 2004-2014

OpenHands

Unsustainable solutions do not count

Velocity is dead

Blog: Velocity is dead: AI-generated compilers and the future of software

Experiments in **quantity**

Experiments in **quantity**

100K line compiler
1 million line web browser
1.6 million line spreadsheet

Experiments in **quantity**

100K line compiler
1 million line web browser
1.6 million line spreadsheet

Zero users

We need experiments in **quality**

OpenHands

We can have nice things

"I've spoken to half a dozen or more different technical coaches in the past couple of weeks, all of whom I know were excellent engineers using TDD in the before-times, and have all told me these days they write almost no code by hand"

Emily Bache

Samman Sociaty

OpenHands

"Whenever someone delivers code, they are shipping their quality process. It doesn't matter if they are coding by hand or using AI."

Bryan Finster

MinimumCD

“The reason I push so hard for CD is that it's the most effective method I've ever seen for uncovering and forcing the improvement of systemic problems that cause pain to our users and ourselves.”

Bryan Finster

MinimumCD


MinimumCD.org
Test
Commit
Repeat

“XP practices are the missing piece for AI-assisted development”

Paul Hammond

Feedback-Driven Development

github.com/citypaul/.dotfiles

Paul Hammond's agent skills for test-driven development, functional design, etc...

How?

OpenHands

Six words

OpenHands

Tell your agent how to test

OpenHands

Example
Lisp Interpreter

Lisp interpreter with GUI

- “Ralph Wiggum” one-shot
- OpenHands CLI --headless
- Opus 4.5 model

▒▒ RALPH LISP ▒▒

▶ Run

EDITOR

```
; Ralph Lisp
(define (factorial n)
  (if (<= n 1) 1
    (* n (factorial (- n 1)))))

(display "10! = ")
(print (factorial 10))

(define (map-demo)
  (map (lambda (x) (* x x))
       '(1 2 3 4 5)))

(print (map-demo))

```

OUTPUT

▒▒ RALPH LISP ▒▒

▶ Run

EDITOR

```
1   ; Ralph Lisp
2   (define (factorial n)
3     (if (<= n 1) 1
4       (* n (factorial (- n 1)))))
5
6   (display "10! = ")
7   (print (factorial 10))
8
9   (define (map-demoooo)
10    (map (lambda (x) (* x x))
11         '(1 2 3 4 5)))
12
13  (print (map-demo))
14
```

OUTPUT

```
10! = 3628800
```

ERRORS

```
[runtime] line 13, col 9: Unbound variable: map-demo
```

Well-tested

Overall Coverage: 86.62%

Coverage by file:

- **env.ts** — 100% ✅
- **errors.ts** — 100% ✅
- **printer.ts** — 100% ✅
- **parser.ts** — 97.76%
- **stdlib.ts** — 97.35%
- **runner.ts** — 95.12%
- **lexer.ts** — 95.2%
- **evaluator.ts** — 89.78% (main interpreter logic)
- **types.ts** — 0% (type definitions only)
- **ui.ts** — 0% (UI not covered by unit tests)

Prompts

Design `**docs/SPEC.md**` with a plan to make a Lisp interpreter for a useful subset of Scheme. It should be implemented in **modular typescript** and build to a **single page html** with inline js and css.

That page should have simple GUI with editor in Code Mirror with syntax highlighting.

Output and errors should display when Run is clicked. Source errors should display inline where possible.

Make it look **cool and hacker-like**.

Populate `**plan/01-init/PLAN.md**` with an implementation plan.

Use `**- [ ]` task bullets**, broken into **Milestone sections**. Milestones should be well-sequenced and have clear deliverables.

There should be data-driven tests of all interpreter features. yaml with input/output saved in `**test-data/*.yaml**`.

UI should be tested using **Playwright**. There should be a makefile, with `make test` `**make check**` (the latter running test, lint, typecheck).

If you feel the SPEC should be updated for clarity, you may.

Update **Tech Guidelines section** in `plan/01-init/RALPH.md` with short notes on your tech decisions and testing strategy.

OpenHands

Tell your agent how to test

OpenHands

Live Example

Blog app

github.com/realworld-apps/realworld

OpenHands

Shift Left

OpenHands

“Back pressure”

OpenHands

Build Time
Test Time

OpenHands

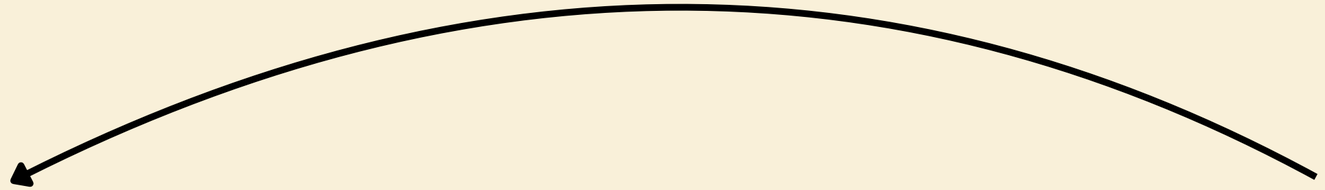
Design Time

Build Time

Test Time

Design	Build	Test
Hexagonal Architecture	Type-Safety	Test-Driven Development
Domain-Driven Design	Linting	Coverage / Mutation Testing
Pure Functions	Code health metrics	Property Testing
Formal Specification	Exhaustive pattern matching	Approval Testing

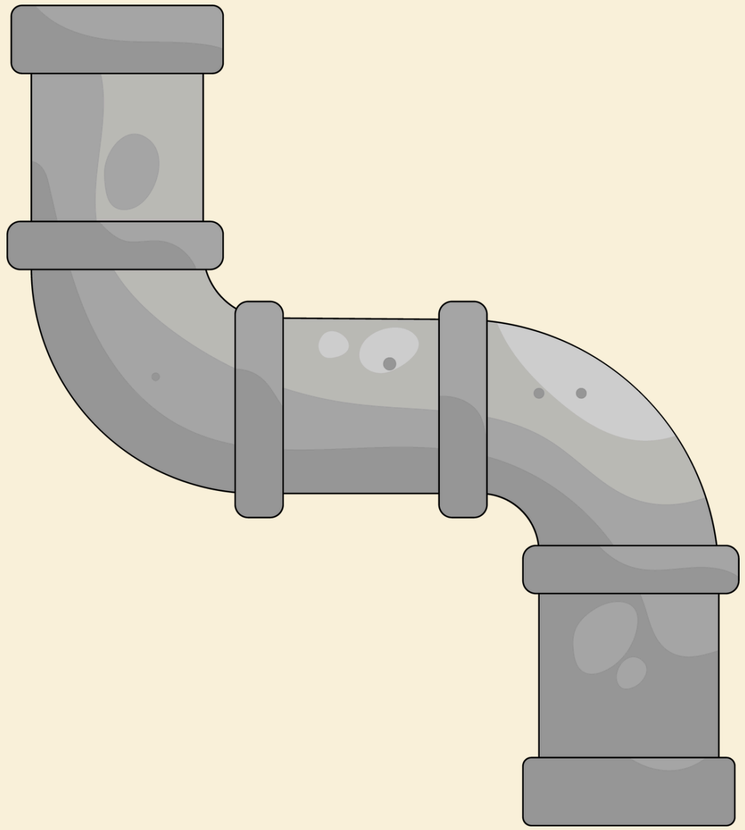
Testability



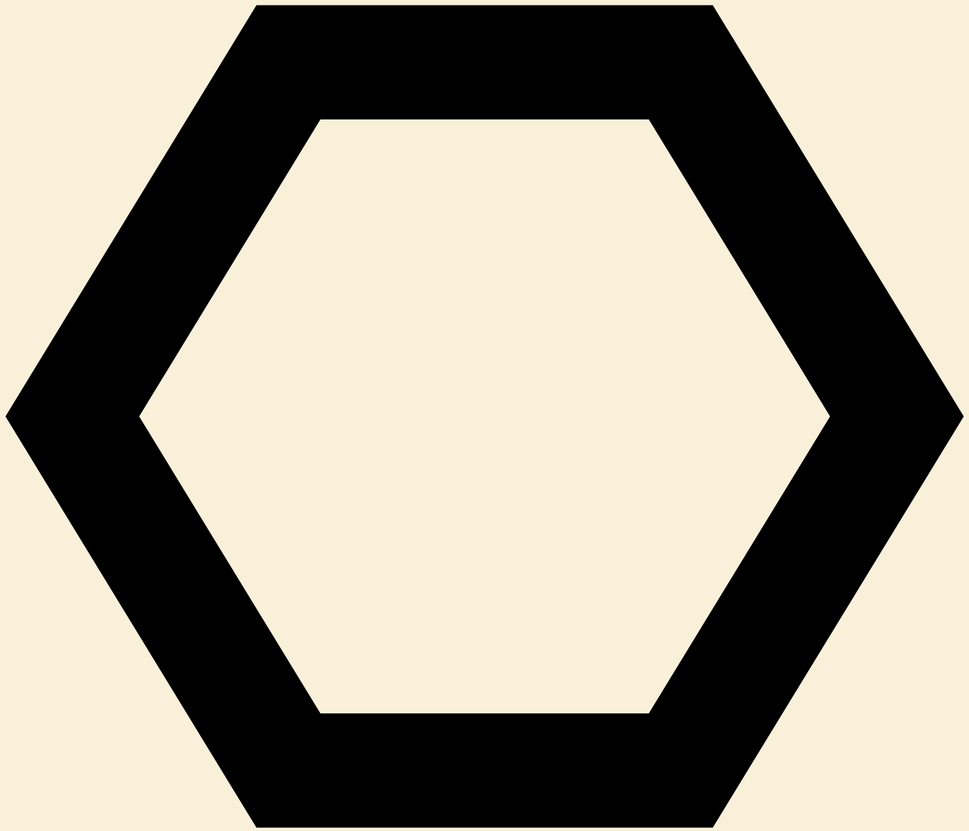
Design	Build	Test
Hexagonal Architecture	Type-Safety	Test-Driven Development
Domain-Driven Design	Linting	Coverage / Mutation Testing
Pure Functions	Code health metrics	Property Testing
Formal Specification	Exhaustive pattern matching	Approval Testing

The octopus problem

In



Out



Hexagonal architecture

OpenHands

Example
Quint Specs

Quint

Executable specs for reliable systems

Feel more confident about your code (human written or AI-generated)

Get Started 5-min guide

Executable

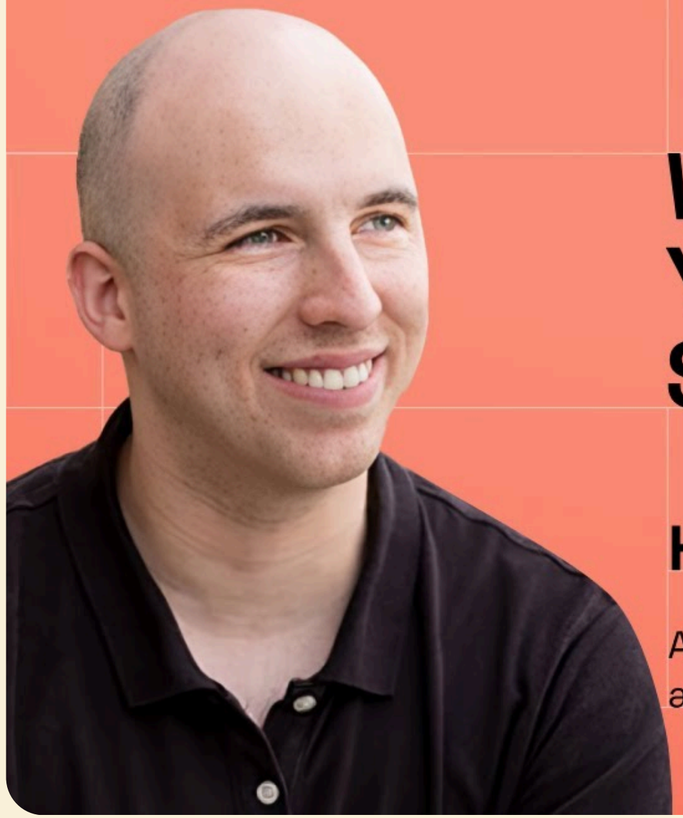
Quint	English & Markdown
✓ checked names and types	✕ not checked
✓ executable	✕ not executable

Abstract

Quint	Programming Languages
✓ define only what matters	✕ define how things happen, in detail

Modern

Quint	Existing Spec Languages
✓ familiar syntax	✕ math-y syntax
✓ CLI and your editor	✕ old GUI tools



SYSTEMS DISTRIBUTED '25

What Isn't Your System Supposed To Do?

Hillel Wayne

Author of "Logic for Programmers" and "Practical TLA+"

OpenHands

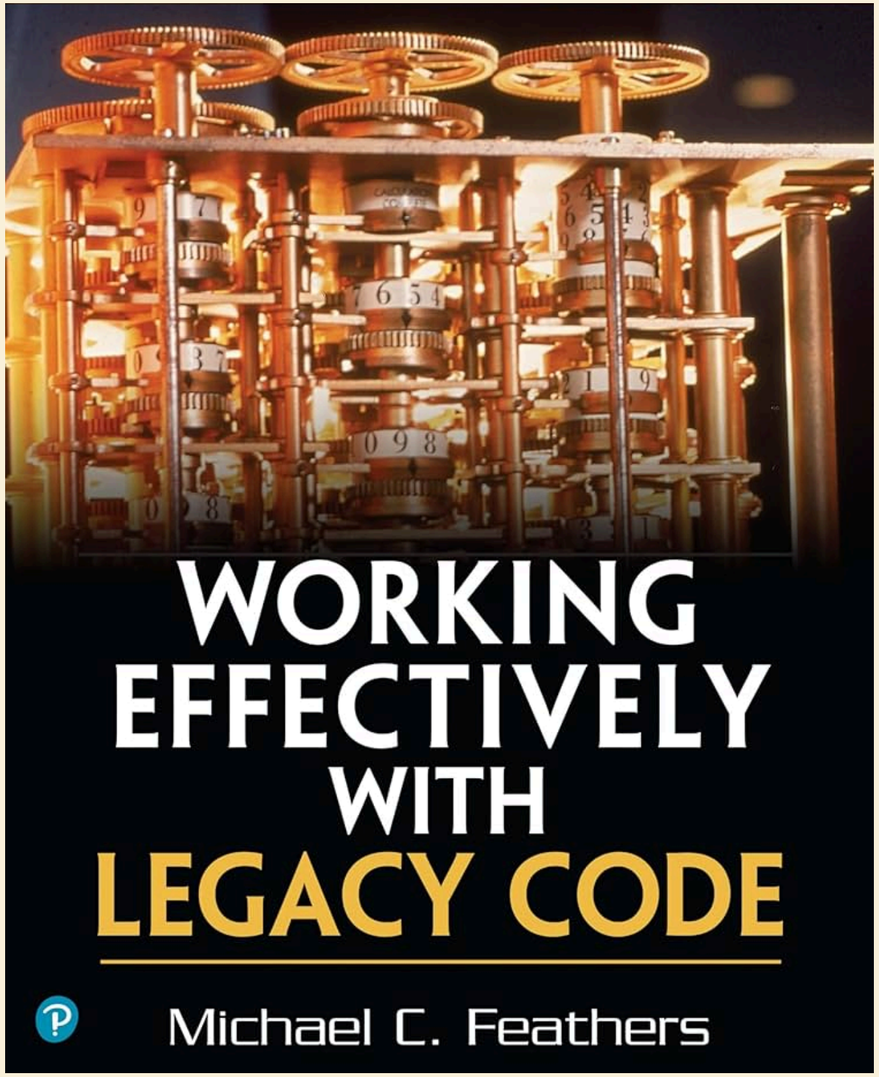
Live Example Revisted

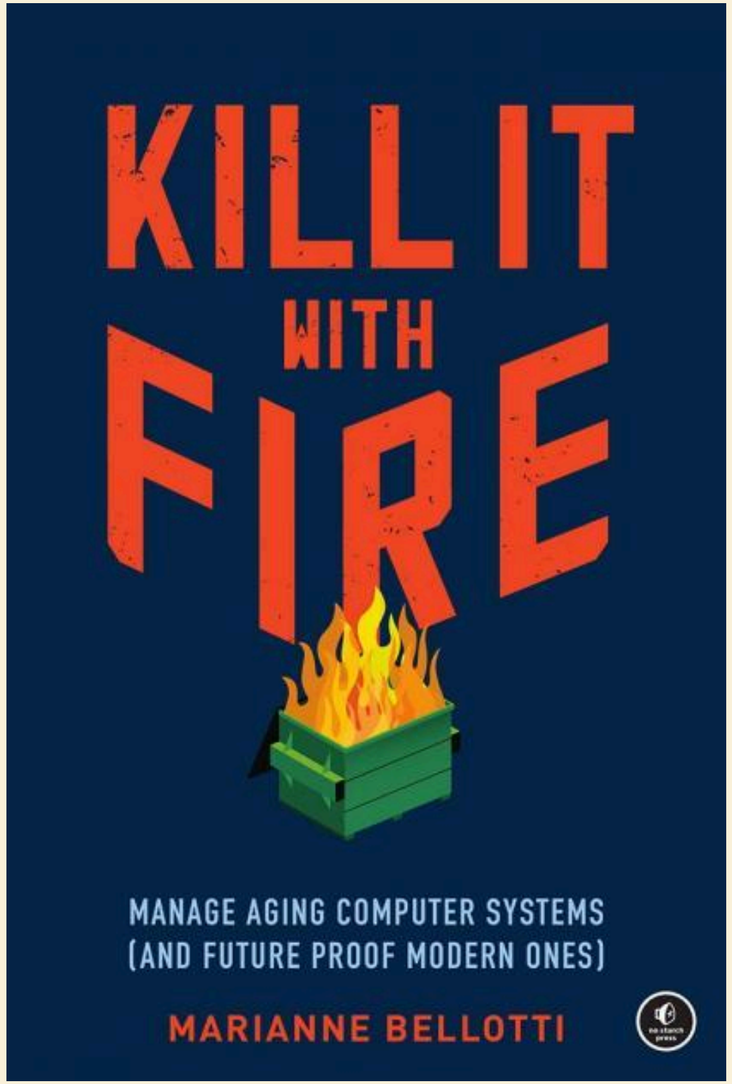
github.com/realworld-apps/realworld

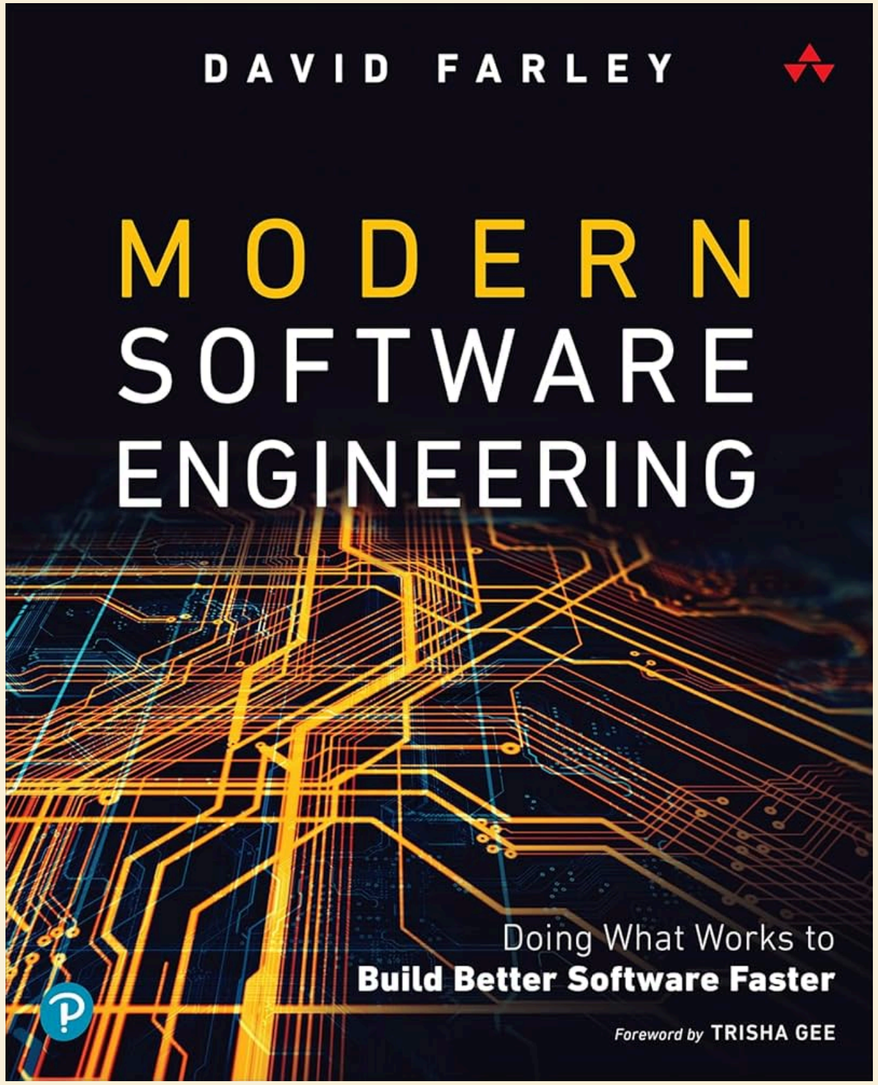
OpenHands

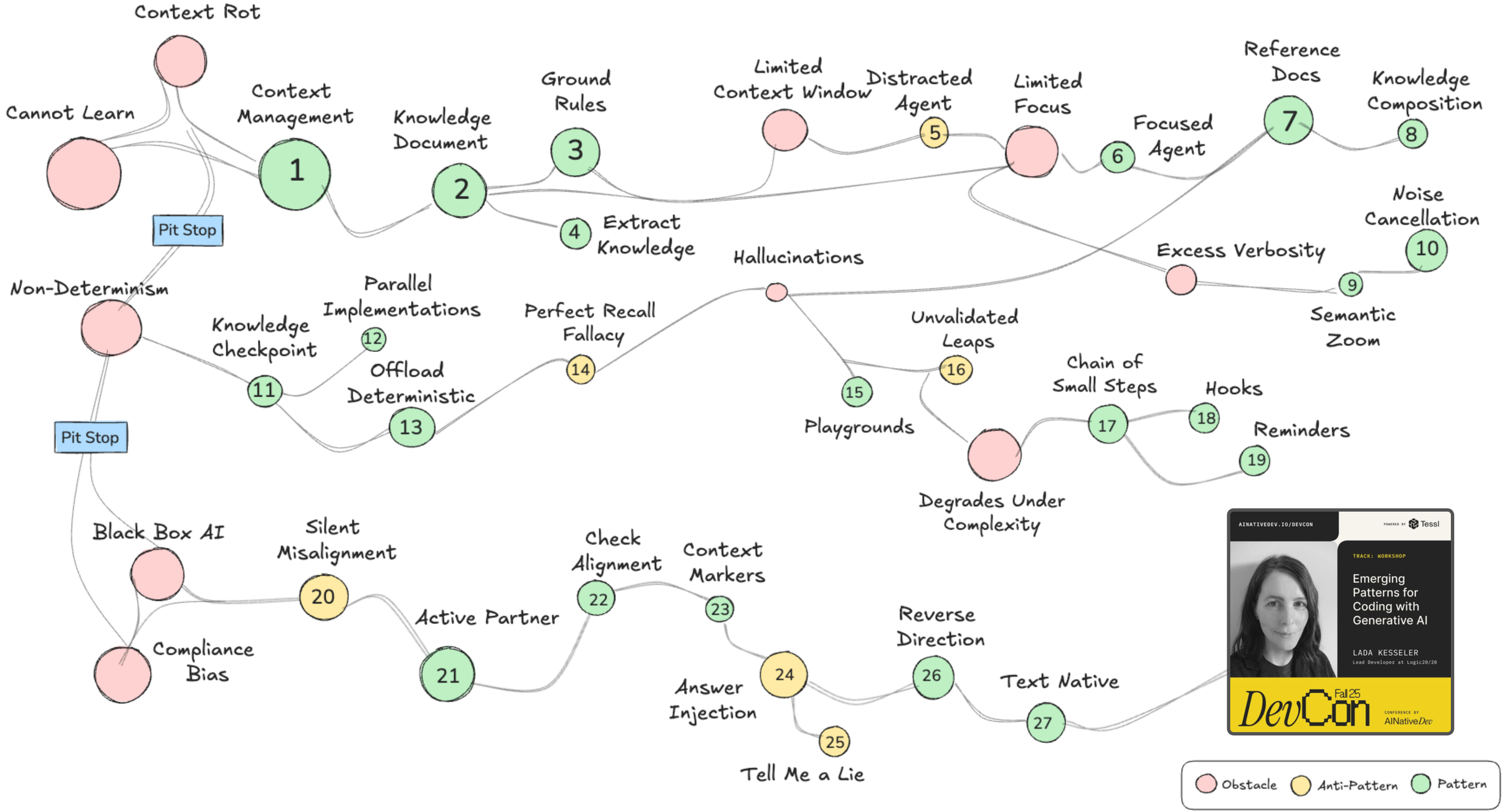
Tell your agent how to test

Resources


WORKING
EFFECTIVELY
WITH
LEGACY CODE
Michael C. Feathers


KILL IT
WITH
FIRE
MANAGE AGING COMPUTER SYSTEMS
(AND FUTURE PROOF MODERN ONES)
MARIANNE BELLOTTI


DAVID FARLEY
MODERN
SOFTWARE
ENGINEERING
Doing What Works to
Build Better Software Faster
Foreword by TRISHA GEE


Context Rot
Cannot Learn
Context Management
1
Knowledge Document
2
Ground Rules
3
4
Extract Knowledge
Limited Context Window
Distracted Agent
5
Limited Focus
6
Focused Agent
Reference Docs
7
Knowledge Composition
8
Noise Cancellation
10
Excess Verbosity
9
Semantic Zoom
Pit Stop
Non-Determinism
Parallel Implementations
12
Knowledge Checkpoint
11
Offload Deterministic
13
Perfect Recall Fallacy
14
Hallucinations
Unvalidated Leaps
16
15
Playgrounds
Chain of Small Steps
17
Hooks
18
Reminders
19
Degrades Under Complexity
Pit Stop
Black Box AI
Compliance Bias
Silent Misalignment
20
Active Partner
21
Check Alignment
22
Context Markers
23
Answer Injection
24
25
Tell Me a Lie
Reverse Direction
26
Text Native
27
AINATIVEDEV.IO/DEVCON
POWERED BY Tessl
TRACK: WORKSHOP
Emerging Patterns for Coding with Generative AI
LADA KESSELER
Lead Developer at Logic20/20
DevCon Fall 25
CONFERENCE BY AINativeDev
Obstacle
Anti-Pattern
Pattern

Agent Benchmarks

OpenHands Index

EvoClaw

Q2 OpenHands Pilot Program

Level up to agentic development at scale in 30 days or less

Who's a fit?

- Multi-user teams
- Mixed use cases
- High accuracy needs
- Multi-agent ready

What you get

- Expert agent config
- Dedicated Slack
- No-cost evaluation
- Post-program discounts

What you give

- 1 month commit
- Bi-weekly meetings
- Eng-to-FDE feedback
- Case study

https://openhands.dev/contact

Questions